

# Some medium-size language models built with LSTM Neural Networks

Stephen Taylor

ZCU

28. dubna 2026

# Outline

Introduction

Examples

Shakespeare

Tolstoy

References

# Intro

- ▶ These slides are a quick overview of some of the language models presented by Andrej Karpathy in "The Unreasonable Effectiveness of Recurrent Neural Networks" which went up on the web in 2015.
- ▶ You might recognize Karpathy's name, because he just released another website in February, in which he presents a 200-line python program which contains all the basics of a LLM, except for sufficient efficiency to make it actually useful.

- ▶ Back then, he was describing a technique that many of us hadn't tried yet, neural networks that included neurons that could be given sticky values in the course of training and execution.
- ▶ LSTM neural networks were introduced in a 1997 article by Sepp Hochreiter and Jürgen Schmidhuber.

## Summary

The web site offered examples of character-based language models built with LSTMs, and the source code for the untrained neural network. Lots of people were excited by this introduction.

# Generating Shakespeare

PANDARUS:

Alas, I think he shall be come approached and the day  
When little strain would be attain'd into being never fed,  
And who is but a chain and subjects of his death,  
I should not sleep.

Second Senator:

They are away this miseries, produced upon my soul,  
Breaking and strongly should be buried, when I perish  
The earth and thoughts of many states.

DUKE VINCENTIO:

Well, your wit is in the care of side and that.

Second Lord:

They would be ruled after this chamber, and  
my fair nudes begun out of the fact, to be conveyed,  
Whose noble souls I'll have the heart of the wars.

Clown:

Come, sir, I will make did behold your worship.

VIOLA:

I'll drink it.

# Commentary

PANDARUS:

Alas, I think he shall be come approached and the day  
When little strain would be attain'd into being never fed,  
And who is but a chain and subjects of his death,  
I should not sleep.

- ▶ So this is a single line of dialog.
- ▶ It begins with a characters name, followed by a colon.
- ▶ 'PANDARUS' isn't made up; he is a character from the Shakespeare play *Troilus and Cressida*.
- ▶ The lines are formatted 'ragged-right', with some white space on the right side of the page.

## More...

PANDARUS:

Alas, I think he shall be come approached and the day  
When little srain would be attain'd into being never fed,  
And who is but a chain and subjects of his death,  
I should not sleep.

- ▶ There is a strange word here, *srain*, but it's possible that Shakespeare's vocabulary is different than ours 400 years later.
- ▶ The syntax is close to right, and the individual clauses of this run-on sentence almost make sense – but not together.
- ▶ The formatting is near-perfect, but the language is only a little better than the 9-character n-gram model.





## generating Tolstoy

```
tyntd-iafhatawiaoihrdemot lytdws e ,tfti, astai f ogoh eoase rrranbyne 'n  
plia tklrgrd t o idoe ns,smtt h ne etie h,hregtrs nigtkie,aoaenns lng
```

This is a sample generated by a model starting to learn how to generate some Tolstoy. It is being trained on *War and Peace* but is obviously not completely trained yet. This sample is generated after the model has been trained for 100 - iterations, which I understand to mean 100 parameter updates. I think that the code batches updates, that is accumulates a sequence of batch-size characters (maybe batch-size = 100) and then considers what changes to make to the parameters to make it more likely that we'll generate this sequence another time. Since the later words are influenced by the earlier ones, it does back-propagation through time, that is, keeps track of the values of the parameters at each time step, as well as the usual back-propagation in space. At this point it looks a bit like a unigram character model. We have plausible numbers of spaces, t and e look like the most common letters.

# generating Tolstoy

```
"Tmont thithey" fomesscerliund  
Keushey. Thom here  
sheulke, anmerenith ol sivh I lalterthend Bleipile shuwy fil on aseterlome  
coaniogennc Phe lism thond hon at. MeiDimorotion in ther thize."
```

This sample is generated after the model has been trained for 300 - iterations.

The distribution of spaces is pretty good, and periods seem to be under control.

It looks like we could pronounce most of these words, even if only *here*, *I*, and *in* are real English words. The length of lines is still strange.

## generating Tolstoy

we counter. He stutn co des. His stanted out one ofler that concossions and  
to gearang reay Jotrets and with fre colt off paitt thin wall. Which das st

This sample is generated after the model has been trained for 500 iterations.

The distribution of spaces is pretty good, and periods seem to be under control.

We now see a number of short but real English words. Some short word combinations start to be realistic, like *and with* and *and to*

## generating Tolstoy

Aftair fall unsuch that the hall for Prince Velzonski's that me of  
her hearly, and behs to so arwage fiving were to it beloge, pavu say  
how, and Gogition is so overelical and ofter.

This sample is generated after the model has been trained for 700 iterations.

Now there are more and longer words. *Prince Velzonski's* is a three-word plausible construction. Lots of four-letter words are still nonsense.

## generating Tolstoy

"Kite vouch!" he repeated by her door. "But I would be done and quarts, feeling, then, son is people...."

This sample is generated after the model has been trained for 1200 iterations.

Almost all of these are real words, and the word-bigrams feel good. The punctuation seems to be perfect, with the quotes balanced, capitalization at the beginning of sentences, and the trailing ellipsis.

## generating Tolstoy

"Why do what that day," replied Natasha, and wishing to himself the fact th  
princess, Princess Mary was easier, fed in had oftened him.  
Pierre aking his soul came to the packs and drove up his father-in-law wome

This sample is generated after the model has been trained for 2000 iterations.

Not much progress over the last bit of training... still some made-up words *oftened* and *aking*, some confusion about the right gender for Natasha.

But Pierre and Natasha are appropriate comrades, and what about that father-in-law ...

I'll stop here... mostly because Karpathy did. We know from the 'finished' Shakespeare generation that this model is not likely to replace the novel.

## Some invisible models

Karpathy's website also showed models for latex and C programs. Both of these have demanding syntax, and the models generate programs that look great, but make about as much sense as the English we've seen – not a very attractive goal for vibe programming.

Getting back to Shakespeare, the results are probably satisfying primarily because we have already a little difficulty in reading the bard.

You could probably write a python program that formatted the output of a 9-gram character model as beautifully as Karpathy's model does in half the time it took to train the model.

## Short summary

So, the language models do train, and they potentially have better results than character n-gram models. It turns out that using them for recognition works better than generation. For example, training LSTMs for sentiment recognition is still plausible , even though this is also a job that an LLM does quite well, because as a medium-size model, the LSTM will need quite a bit fewer resources to execute than the LLM. (at least last week. Maybe not next week.)



# References



KARPATHY, Andrej. *microgpt*.

<https://Karpathy.github.io/2026/02/12/microgpt>



KARPATHY, Andrej. *The Unreasonable Effectiveness of Recurrent Neural Networks*

<https://Karpathy.github.io/2015/5/21/rnn-effectiveness>